C-MCPN: The Clingo McCulloch-Pitts Network Editor

An Answer-Set Programming Framework for Artificial Neural Networks

David Gonzalez[1], Joshua T. Guerin, Ph.D.[2]
Mathematical Sciences
University of Northern Colorado
Greeley, CO 80639
[1]********@bears.unco.edu
[2]********@unco.edu

Abstract

As Artificial intelligence and generative AI continue to engage the public in additionally profound ways, there is growing interest in developing more efficient underlying technologies. In this paper we explore the use of efficient Answer Set Programming (ASP) languages to model and conduct inference over artificial neural network (ANN) architectures. This combination of technologies is not trivial, so we also introduce the C-MCPN (Clingo McCulloch-Pitts Network) Framework and Editor to support our work in generating more efficient neural networks. The C-MCPN Framework provides a novel representation for encoding ANN structure and inference in ASP, while the C-MCPN Editor is a working, accessible tool that makes this representation practical to construct, visualize, and experiment with. Together, these establish a foundation for future work exploring whether ASP-based learning models can offer meaningful efficiency gains over traditional ANN implementations, an open question we are actively pursuing.

1 Introduction

Artificial Intelligence (AI) systems and their applications have been extremely consequential on the modern world over the decades. This has been especially evident over the last decade with skyrocketting awareness and use of generative AI (GEN-AI) techniques. In particular, these include techniques from

machine learning (ML) like the artificial neural networks (ANNs) that power contemporary large language models (LLMs).

These AI systems have shown great promise, but the underlying technologies come with significant drawbacks. Concerns include the growing footprint of data centers, which has garnered public interest [7]; the worsening use of finite resources such as fresh water and electricity [5]; and the potential for substantial environmental and health impacts [4].

Conversely, Answer Set Programming (ASP) languages live at a different end of the research spectrum in terms of their efficiency. They are designed to solve NP-Complete and NP-Hard problems in a fast and efficient manner on limited hardware. Our current work involves exploring the use of ASP languages to accelerate the processes of learning and inference. With this, we hope to reduce the resources needed for these technologies.

In this paper we introduce the `C-MCPN Editor`: The *Clingo McCulloch-Pitts Network Editor*. The C-MCPN Editor is a tool we developed to assist in the exploration of ASP-based networks. It allows a user to model the network architecture, demonstrating that ANNs, specifically McCulloch-Pitts Networks (MPNs), can be used to model and conduct inference over McCulloch-Pitts neurons [6].

2 Background

Our work is related to the merging of ANN technology with more efficient Answer Set Programming languges. In this section we will provide a brief background on both subjects, motivating our work on C-MCPN.

2.1 Artificial Neural Networks (ANNs)

ANNs were originally inspired by the structure and function of biological neruons. Their basic building blocks, the artificial neurons, can be traced back to the work of McCulloch and Pitts in the 1940s [6]. These early models mimicked the behavior of biological neurons by using a simple thresholding function that determines whether a neuron will fire or not. The name for the C-MCPN was inspired by this early work, though its implementation is not a pure McCulloch-Pitts model.

The neurons can be connected together to form a network, whose structures can vary widely from simple knowledge graphs to feedforward networks to more complex recurrent architectures [11]. For our work, the feedforward neural network was chosen for its simplicity and wide use in the field. It consists of layers of interconnected neurons that process data in a single forward direction [3], shown in Figure 1. These models can learn through a training process where

weights are adjusted based on error, but notably their underlying structure is similar to that of the original McCulloch-Pitts model.

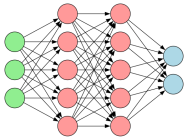

Figure 1: A diagram of a simple feedforward neural network

2.1.1 Environmental Impacts

The introduction of big data, chiefly by scraping the broader Internet, changed the landscape for AI. It allowed for statistical-based models to flex their strength, which require massive amounts of data for training purposes [3]. This incredible need for information highlights an important drawback in the form of *data efficiency* [1]. The lack of efficiency then translates to a significant amount of necessary resources.

As an example, consider the cost to train a single model, ChatGPT-3. According to Vries [10], GPT-3 reported around 1.3 million kWh *just* to be trained. With some quick, back-of-the-envelope calculations, we can put this into perspective. In 2022 the average American household used approximately 10,791 kWh [9], and by dividing these out we find that the model's energy use was proportional to:

$$1{,}300{,}000 \text{ kWh} \times \frac{1 \text{ household}}{10{,}791 \text{ kWh/year}} \approx 120.5 \text{ households.}$$

Though this is a remarkably over-simplified example, it puts into perspective how the technology is already unsustainable. This highlights the dire need for more efficient, sustainable AI and ML technologies [10], an open problem yet to be solved.

2.2 Answer Set Programming (ASP)

Answer Set Programming serves as an amalgamation of Prolog-style quantified logic with powerful advances in satisfiability (SAT) solvers that occurred throught the 1990s and into the 2000s. In this section we will discuss the basic syntax and semantics of the ASP language, Clingo, which serves as a technological foundation of C-MPCN [2].

2.2.1 Syntax and Semantics

The atomic units in Clingo are *literals*, which describe ideas and *facts* that serve as statements that are held to be true. The basic syntax is the application of a *property* to a literal, `property(atom)`. E.g.,

```
red(apple).
```

is a complete statement (and program), attributing a property of red to an idea named `apple`.

Further knowledge is*inferred* using aggregate statements known as *rules*, taking the form: `consequent :- antecedent`. This mirrors an *implication* from formal logic, as the `:-` symbol is read equivalently to $\leftarrow$. In such a statement the consequent *must* be true whenever the antecedent is true. E.g.,

```
eat(Fruit) :- red(Fruit), ripe(Fruit).
```

describes a preference over what fruit will be eaten. I.e., "If a fruit is red and ripe, then we eat the fruit."

In this case rather than the literal `apple`, we use a *variable* to allow for inference. The `Fruit` can be *any* contextually-relevant literal in the program, so long as it possesses the required properties.

3 Methodology

Combining the efficiency of ASP languages with the representational power of ANNs is a non-trivial task. Not only does it require a novel representation of the ANN structure, but Clingo has no floating-point support. This requires clever simplifications of the activations functions and gradiant descent to use them over integers. Additionally, large ANNs can cause combinatorial explosions in the search space, making grounding the programs difficult. The C-MCPN Framework and Editor were developed to address some of these challenges, providing a structured way to represent and work with logical networks in Clingo.

3.1 Domain Selection

To build and test the framework the illustrative domain of digital logic for hardware development was selected. Its simplicity and familiarity make it an ideal domain for experimentation.

For our example network, a 1-bit adder seen in Figure 2, we used neurons with three types of well established logic gates: AND, OR, and XOR. In Table 1 we show the neuron definitions of these gates, and their truth tables. Using a

combination of these and more gates, one is able to construct complex logical operations[1].

Table 1: Neuron Definitions and Truth Tables.

AND $g(\vec{x}) = x_1 \cdot x_2$			**OR** $g(\vec{x}) = x_1 + x_2$			**XOR** $g(\vec{x}) = \lvert x_1 - x_2 \rvert$		
x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$
T	T	T	T	T	T	T	T	F
T	F	F	T	F	T	T	F	T
F	T	F	F	T	T	F	T	T
F	F	F	F	F	F	F	F	F

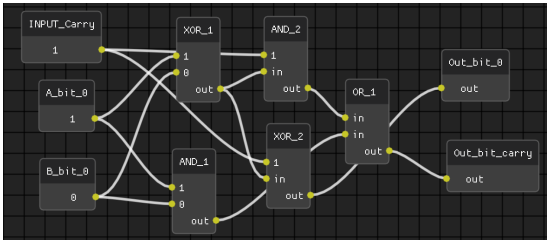

Figure 2: A screenshot of a 1-bit adder network in the C-MCPN Editor.

3.2 Design Approach

The representation of an artificial neuron requries both the *structure* designation, and the function used to *activate* it. When activated the neuron will propagate a signal to its output given the provided input signals. Its value then further propagates through the network along edges and activating other neurons.

An `AND` gate is used to illustrate how we represent an artificial neuron:

```
% gate(TYPE, OUTPUT, INPUTS)
gate("AND", Out, In1, In2) :- values(In1), values(In2),
                              Out = (In1 * In2).
```

The *structure* of the neuron is indicated by the parameters: in the case of an `AND` gate this requires two input parameters and a single output. The

[1] An elegant and simple proof of Clingo's Turing completeness falls out of the C-MCPN Framework.

activation function is encoded directly in the rule structure using the arithmetic equivalent provided in Table 1.

The neuron *connectivity* specified in the gate definitions can then be used to forge connections of a network. A connection from the 1-bit adder sample network is built as:

```
% neuron(type, unique_id)
neuron("INPUT", "INPUT_1").
neuron("XOR", "XOR_1").

% edge(source_id, target_id)
edge("INPUT_1", "XOR_1").

% val(neuron_id, value).
val("INPUT_1", 1).
```

Finally, the value inference program is used to store the possible values, activate the neurons, and propagate the values through the network. The program for a 2-input gate can be built as:

```
%% Value domain
values(0;1).

% Edges carry values from input to output
edge(In, Neuron_id, Val) :-
    edge(In, Neuron_id),
    neuron(_, Neuron_id),
    neuron(_, In),
    val(In, Val).

% 2 input gates
val(Neuron_id, Val) :-
    gate(Type, Val, Val1, Val2),
    neuron(Type, Neuron_id),
    edge(In1, Neuron_id, Val1),
    edge(In2, Neuron_id, Val2),
    In1 != In2.

%% Each neuron has one value
{ val(Neuron_id, Val) : values(Val) } 1 :- neuron(_, Neuron_id).

%% Show values
#show val/2.
```

The rule carrying the values along the edges is vital to the functioning of the network. Without it, the network gets "stuck" and is unable to infer what the values should be. This is a key insight we had during development.

4 C-MCPN

To better explore the ANNs implemented within Clingo we developed the C-MCPN software package to automate our workflow. The system itself is designed around a standard (unix-like) terminal use and is extended into a graphical user interface (GUI).

The core functionalities of C-MCPN are built on the ASP language Clingo, as described in the preceding sections. The editor and interface were constructed in Python using the DearPyGui library, allowing us to take advantage of its node-based editing capabilities.

For node reorganization in the editor we used a modified version of the Sugiyama algorithm for layered graph drawing, taking just the layer assignment step and a simplified node positioning step [8].

4.1 C-MCPN Framework

From what the authors found in the literature, the C-MCPN Framework is a novel approach to neural network representation within a logical programming paradigm like ASP. The networks themselves have relatively simple representations that can be leveraged to create complex operations. The framework, which can be seen in Figure 3, uses three main program types, all of which are contained in the `C-MCPN/` directory of the project. The content in the gate definitions program defines the behavior of each gate, but every default gate we included also has a special header used to send meta-information to the C-MCPN Editor.

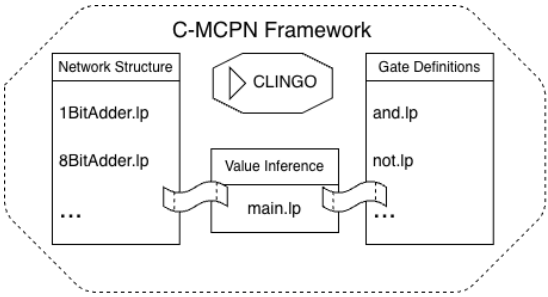

Figure 3: A visualization of the programs involved in the C-MCPN Framework.

4.2 C-MCPN Editor

The `app/` directory of the project contains the source code for the C-MCPN Editor, which can be thought of as an interactive development environment

(IDE) for logical networks. Its main features include the ability to add/remove neurons, connect/disconnect them, copy/paste them, and run inference on the network. Additionally, the editor provides a live preview of the ASP code generated from the network structure, allowing users to see how their changes affect the underlying logic program.

User created networks are saveable to JSON and ASP format, and loadable from JSON format. This facilitates creation of complex ANNs, and saving the network to ASP format allows direct use of the C-MCPN Framework from the terminal instead of the IDE.

A screenshot of the editor can be seen in Figure 4, showcasing the sample 1-bit adder network.

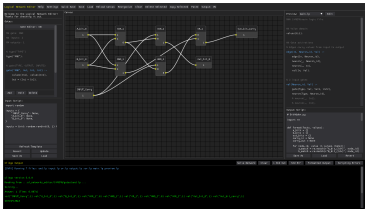

Figure 4: Screenshot of the C-MCPN Editor, showing a sample 1-bit adder network.

4.3 AI-Assisted Development

Claude (Anthropic) was used as a programming assistant throughout development of the editor, helping with DearPyGui module integration, debugging, and some code generation. All design decisions, research direction, and domain-specific logic (ASP, gate definitions, network structure) were conscious decisions by the authors. Each line of code generated by Claude was reviewed, edited, and approved by the authors before being added to the codebase. In order to maintain a supervisory-level of control the agent was not given direct access to the codebase or the repository. The files that Claude had input in are contained in the `app/` directory of the project, allowing users the ability to inspect these files if they so desire.

Use of Claude was employed only in the above described sections in order to accelerate the development process under circumstances of limited resources.[2]

[2]The authorship of this paper is without the assistance of any AI system, including planning, writing, editing, and repository maintenance.

5 Results

The result of this work is the C-MCPN Framework and Editor, a novel approach to representing and reasoning over logical networks within logic programming. The framework and its neuron gates were validated using an 8-bit adder network, constructed and visualized with the editor. Results on efficiency and scalability, our primary motivation, are left for future work, as they matter more for the adaptive, learning-capable network framework we are currently developing. Both tools are open source and available at `https://github.com/David-Gonzalez-Sua/logical_networks/`.

5.1 Limitations

This work is a proof of concep, and as such has some limitations. The framework lacks cycle detection and has not been tested on recurrent networks, which is likely to cause issues for the value inference program. It has has also not been tested on large networks, and its performance at scale is unknown.As noted in Section 3, Clingo does not have native support for floating point numbers, limiting potential applications.

5.2 Future Work

Our current work focuses on using the C-MCPN Framework as the skeleton for an adaptive, learning-capable neural network. This framework is in its early stages, but we hope to add it as a feature in the C-MCPN Editor.

6 Conclusion

The growing resource demands of modern AI motivate the search for more efficient alternatives, and Answer Set Programming presents a promising, underexplored direction. The C-MCPN Framework and Editor are a first step toward this goal, providing the representational foundation needed to build and study ASP-based neural networks. As we continue this work, our hope is that ASP's natural efficiency advantages will translate into meaningful improvements to the efficiency of learning and inference processes.

References

[1] Andrew Cropper, Sebastijan Dumančić, and Stephen H. Muggleton. "Turning 30: New Ideas in Inductive Logic Programming". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. 2020, pp. 4833–4839. DOI: http://dx.doi.org/10.24963/ijcai.2020/673.

[2] Martin Gebser et al. "Multi-shot ASP solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82. DOI: http://dx.doi.org/10.1017/S1471068418000054.

[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[4] Yuelin Han et al. *The Unpaid Toll: Quantifying and Addressing the Public Health Impact of Data Centers*. 2025. DOI: http://dx.doi.org/10.48550/arXiv.2412.06288.

[5] Pengfei Li et al. *Making AI Less "Thirsty": Uncovering and Addressing the Secret Water Footprint of AI Models*. arXiv:2304.03271. 2023. DOI: http://dx.doi.org/10.48550/arXiv.2304.03271.

[6] Warren S. McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133. DOI: http://dx.doi.org/10.1007/BF02478259.

[7] Engineering National Academies of Sciences and Medicine. *Implications of Artificial Intelligence–Related Data Center Electricity Use and Emissions: Proceedings of a Workshop*. Ed. by Anne Frances Johnson and Kasia Kornecki. Washington, DC: The National Academies Press, 2025. DOI: http://dx.doi.org/10.17226/29101.

[8] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. "Methods for Visual Understanding of Hierarchical System Structures". In: *IEEE Transactions on Systems, Man, and Cybernetics* 11.2 (1981), pp. 109–125. DOI: http://dx.doi.org/10.1109/TSMC.1981.4308636.

[9] U.S. Energy Information Administration. *How much electricity does an American home use?* U.S. Energy Information Administration, Frequently Asked Questions. 2024. URL: https://www.eia.gov/tools/faqs/faq.php?id=97&t=3.

[10] Alex De Vries. "The growing energy footprint of artificial intelligence". In: *Joule* 7.10 (2023), pp. 2191–2194. DOI: http://dx.doi.org/10.1016/j.joule.2023.09.004.

[11] Jie Zhou et al. “Graph Neural Networks: A Review of Methods and Applications”. In: *AI Open* 1 (2020), pp. 57–81. DOI: http://dx.doi.org/10.1016/j.aiopen.2021.01.001.